

n the same notification thread, that comment will automatically be added to the incident on Monte Carlo.

Our goal is to be able to integrate Monte Carlo with GitLab so that whenever we get an alert on Slack, a triage issue would automatically be opened on GitLab and we'd follow the same Data Triage procedure as usual.

There is detailed information including a video section in the official Monte Carlo documentation on how to respond to an alert.

Incident status

Each MonteCarlo incident has always a status. See the following list when to use which status:

MonteCarlo status	Context
Actions done or to -do	
----- -----	
----- -----	

Fixed	Incident is not active anymore.
Actively worked on	resolving the incident or the incident is normalized automatically.
Expected	Incident was flagged by MonteCarlo correctly. We knew that this was coming, like a batch update or a schema change that was in the works.
None	
Investigating	Actively working on the incident
Root cause investigation and resolve if needed	
No action needed	Incident was flagged by MonteCarlo correctly, but its not a breaking change
None	
False positive	Incident was flagged by MonteCarlo wrongly
None	
No Status	Default status by MonteCarlo
Start investigating and update status	

Providing feedback to the anomaly detection model

The only two ways of influencing Monte Carlo's anomaly detection model are described in Training data section of the Monte Carlo documentation and are:

1. Mark as normal: anomalies are automatically excluded from the set of data that trains models. This isn't a status of the incident per-se, this is a specific functionality you can access by hovering over the alert graph in Monte Carlo
2. Select training data: by interacting with the chart of a monitor, users can exclude periods of data from training models. They can also use exclusion windows to define periods of time that should be ignored for an entire warehouse, database, schema, or table. These can be one-off or set for recurring holidays.

When you mark an alert as "normal" in Monte Carlo, the anomalous data point that triggered the alert will be re-introduced into the training set for the anomaly detection model. This causes the threshold to widen, meaning similar anomalies will not trigger alerts in the future.

The key consequences of this action include:

- Threshold Adjustment: The system's detection thresholds will automatically widen to accommodate the pattern you've identified as normal, reducing alert noise for similar events.
- Model Retraining: The anomaly detection model is retrained with this data point now included as part of the "normal" pattern, which affects future anomaly detection.
- Reduced Alert Noise: You'll receive fewer alerts for similar patterns that were previously flagged as anomalies but are actually expected behavior.

When to Mark Alerts as Normal

You should mark alerts as normal when:

- Expected Business Changes: The detection was a valid anomaly from a statistical standpoint, but was the expected result of something like a pipeline change or planned maintenance. For example, an intentional deletion of data that triggers a volume anomaly alert.
- Non-Issues: You don't want to be alerted about similar anomalies in the future because they represent normal business patterns rather than actual data issues.
- Recurring Patterns: For patterns that appear unusual to the system but are actually regular business cycles (like monthly processing jobs that cause volume spikes).

It's important to note that alert statuses ("Expected", "No action needed", "False positive") do not provide feedback to the models that generate thresholds. Thresholds will not change or adjust based on the alert status provided. Model feedback is managed through the "Mark as normal" process.

After marking alerts as normal, it can take several hours before the new, widened thresholds are visible in the system.

For multiple related alerts, you can use the "Mark all as normal" option in the Tune model drop down on the top right of the anomaly chart ([link to docs](#)), to avoid clicking through each event individually.

Note on DWH Permissions

In order for Monte Carlo to be integrated with Snowflake, we have had to run the permissions script as specified in the official docs for each database we needed to monitor.

The same script has to be run as many times as we have databases to monitor (in our case raw, prep and prod) with the correct values for the \$databasetomonitor variable. The scripts foresees new tables to be added to existing schemas. In case of a new schema the script has to be executed again for the database the schema resides. The data observability user is stored on our internal data vault.

Please note this is an exception to our usual permission-handling procedure, where we rely on Permifrost, because observability permissions are an edge-case for Permifrost and not yet supported by the tool.

There is an ongoing feature request on Permifrost for adding granularity to the way permissions are set, but no solution has been agreed on yet.

Monitoring strategy

By default, we monitor all tables in the RAW, PREP, and PROD databases in Monte-Carlo, unless

there is a specific reason not to, or if we reach the limits specified in our contract. Excluded tables or schemas from monitoring are documented below.

Exclude sandbox schemas

Sandbox environments are generally created for the purpose of testing. We normally don't take any actions on them even if any alerts come through in our triage slack channels. For this reason we exclude monitoring schemas that contain sandbox to avoid getting any alerts from them. This has been set via an exclude rule in Monte Carlo.

Notification strategy

All incidents are reported in MonteCarlo incident portal. For triage purposes the most important (which requires action) are routed towards Slack. The following matrix shows per data area which type of monitors are routed and towards which channel:

Database		DataScope		Volume	
Freshness		Schema changes		Custom monitors	
-----		-----		-----	
-----		-----		-----	
RAW		TIER1		data-pipelines	
data-pipelines		analytics-pipelines (once per day)		data-pipelines	
		TIER2		data-pipelines	
data-pipelines		analytics-pipelines (once per day)		data-pipelines	
		TIER3		data-pipelines	
data-pipelines		analytics-pipelines (once per day)		data-pipelines	
PREP		n/a		-	
-		-		-	
PROD		COMMON		analytics-pipelines	
analytics-pipelines				-	
		WORKSPACE		-	
-		-		-	
		WORKSPACE-DATA-SCIENCE		data-science-pipelines	
-		data-science-pipelines		data-science-pipelines	
		LEGACY		-	
-		-		-	

COMMON is also the COMMONRESTRICTED equivalent. It excludes COMMONPREP and COMMONMAR. WORKSPACE-DATA-SCIENCE is the only workspace schema we are including in the notification strategy

Only these two models (snowplowstructuredevents400 and snowplowstructuredeventsall) of the LEGACY schema have been included temporarily as per MR !7049

This notification strategy is the basis for any alert being sent from Monte Carlo towards Slack. However, as of Notifications 2.0 Monte Carlo has introduced Audiences. This means, the above notification strategy has now been migrated towards Audiences and we have the following Audiences in place, sending alerts to Slack channels as specified below:

Audience	Slack Channel
-----	-----

Analytics Engineers	analytics-pipelines	
Analytics Instrumentation	ganalyzeanalyticsinstrumentation	
Data Engineers	data-pipelines	
Data Science	data-science-pipelines	
Sales Analytics	sales-analytics-pipelines	

Domains

We have the availability to use domains in our Monte Carlo environment. Currently domains can be used to create separate environments for separate team members, domains automatically filter monitors and incidents by projects and datasets. We have a limited number of domains available.

Domain	Description	Data Scope	
-----	-----	-----	
Data Platform Team	Domain for the Data Platform Team - scope raw data layer in Snowflake	Snowflake raw layer	

Use domains

In Monte Carlo UI in the top right corner there is a dropdown box available which you can select a particular domain or all domains.

BI Integrations

When we initially deployed Monte Carlo at GitLab, we defaulted to Sisense as a BI tool, as it was what we were using at the time. However, recently we have started migrating to Tableau and therefore we have added the Tableau integration to our Monte Carlo instance.

It is now possible to check table and field lineage from our raw models to Tableau objects, such as Tableau Views, Tableau Live Data Sources or Tableau Extract Data Sources.

The Sisense integration and Tableau integration coexist on Monte Carlo and all related Sisense charts as well as Tableau objects can be seen on the lineage charts.

Internal Monte Carlo handbook page

Additional internal information is available in our Internal GitLab Handbook.

SECTION: enterprise-data/platform/pipelines/_index.md

Overview

Data Pipelines pages are available in our Internal GitLab Handbook.

SECTION: enterprise-data/platform/python-tool-package-management/index.md

On this page the main intention is to keep all details about the inventory we are using for the Data Platform, mainly thinking of the Python libraries and the following tools.

The motivation is to keep a neat inventory list with details and criteria of who, why, when and what we will upgrade. Also, the plan is to expose detailed conditions regarding the upgrade and command line application to gather all details about the recent and latest versions of Python libraries. The inventory list contains:

- · Python versions in use
- · Tools we are using (dbt, airflow, permifrost, meltano)
- · Python libraries (packages) in use (i.e pandas, requests, matplotlib)

We strive to have a DRI for every tool or package. The DRI responsibility is to keep that tool or package in a healthy shape by advice and or initiate upon upgrades.

General motivation for upgrading

As the tool upgrading strategy is specific to each tool, it is up to DRI when the upgrade will happen and it/data-team/ which circumstances.

Motivation for upgrading:

1. Reduce vulnerabilities
1. Fix bugs and crashes
1. Ensure compatibility with other updated technologies
1. New functionality introduced which we intend to use
1. Ensure proper support
1. Easier future upgrades (not falling to far behind on version)
1. Marketplace relevance
1. End of life

End of life check up

At minimum we abide to an end of life policy. This is because anything becomes deprecated usually no longer receives any bug fixes, even critical security ones, which is a security risk to us.

The main benchmark for updating the Python version is the end-of-life parameter. In case, the version is not supported anymore, should be a candidate for upgrading.

Initiate/Schedule a version upgrade

A DRI is an expert in the domain of their tool or package and monitors new versions and releases. If and when an upgrade is applicable, the advise and initiates an upgrade. This advice contains:

1. Motivation for the upgrade
1. Impact and dependencies

1. Criticality of the upgrade (timeline)

Above will be documented in an issue and discussed during a weekly Data Platform Team meeting.

Planning

Upgrades are scheduled as an OKR (P2) on a quarterly basis, following the Data Planning drumbeat. If needed (i.e. in case of a security vulnerability) we can schedule during the quarter as a P1. This will result compromising P2-OKR work that was already scheduled.

Python Image/Container Inventory List

There are several images we are using ·Python. Various versions are in use (≥ 3.7), due to the following reasons:

- use cases are various for the different projects
- some libraries require a specific ·Python version (due to dependencies)
- multiple teams are using images and they have requirements for the specific versions implementation

> Table 1: List of images we are using

Image	Version in use	Image version	DRI
User			
----- ----- -----			
----- ----- -----			
analytics		3.10	N/A
TBA	Data Platform		
airflow-image	3.8	python:3.8	
TBA	Data Platform		
ci-pyhton-image	3.8	python:3.8-slim-buster	
TBA	Data Platform		
data-image	3.10.3	python:3.10.3	
TBA	Data Platform		
dbt-image	3.10.3	python:3.10.3	
TBA	Data Platform		
gitlab-data-meltano	3.8	meltano/meltano:v2.16.1-python3.8	
TBA	Data Platform		
mlfow-infra	3.8	python:3.8	
TBA	Data Scientists		

<details><summary>Dependency graph (Click to expand)</summary>

mermaid

flowchart LR

P38 --> pip

P388 --> pip

```
P310 --> pip
pip --> airflow-image
pip --> ci-python-image
pip --> data-image
pip --> dbt-image
pip --> mlflow-infra
pip --> gitlab-data-meltano
pip --> permifrost
```

```
data-image    --Inherit--> gitlab-data-utils
data-image    --Inherit--> analytics
ci-python-image --Inherit--> analytics
```

```
subgraph Python
  P38[Python 3.8]
  P388[Python 3.8.8]
  P310[Python 3.10.3]
end
```

```
subgraph Package manager
  pip[pip]
end
```

```
subgraph Images
  airflow-image
  ci-python-image
  data-image
  data-science
  dbt-image
  gitlab-data-utils
  mlflow-infra
  gitlab-data-meltano
  permifrost
end
```

```
subgraph DEV [Development environment]
  analytics
end
```

</details>

Approach to update Python version

The current section is the guideline of how and when to upgrade the Python version in the particular image. There is no unified condition of when to upgrade the Python version, most of the listed items are recommendations and best methods. The main reason for this statement is the versatile use cases for the images we are using.

Another vital benchmark for extra upgrading is security vulnerabilities. Source for checking potential Python vulnerabilities:

- cvedetails: Python: Security Vulnerabilities
- readthedocs: Python Security Vulnerabilities

In case of any known and confirmed Python vulnerabilities, we should start a process for upgrading the Python version as soon as possible.

Generally, in the following tex exposed a set of advices and, if there is a particular reason NOT to upgrade the Python version, will be good to expose the explanation.

For maintaining and upgrading Python versions and check when the particular version will become obsolete, check on Python supported timeline, or alternatively can check endoflife.date. The inventory list for the images we are using is listed in the table below. Python version upgrades will be determined from case to case as the impact can be huge on the image. A good option is to consider end-of-life for the particular version.

Example for the end-of-life policy:
Regarding the upgrade policy, think we should at the minimum abide by the end of life policy. This is because anything becomes deprecated usually no longer receives any bug fixes, even critical security ones, which is a security risk to us.

- > 1. Python life cycle, we are currently using the following Python versions which are either at end-of-life or close to it:
- > 1. 3.7 already at end of life
- > 1. 3.8 at end of life in end of 2024

Tools inventory list

> Table 2: List of tools GitLab Data team is using

Tool name			
Version in use Version supported timeline			
How to upgrade DRI Users			
Upgrade Policy			

----- ----- -----			

----- ----- -----			

dbt 1.9.2 link - dbt best practices for upgrading			
dbt version TBA - Data Platform			
Analytics Engineers			
Data Scientists Not more than 2 versions behind (beta release excluded) and minimum support level critical			
airflow 2.5.3 link Upgrade Plan for Airflow			
TBA Data Platform			
released > 1 year			
Current version			
permifrost 0.15.4 link			
Upgrading permifrost version			

@rbacovic Data Platform	Not more than
2 versions behind (beta release excluded)	
meltano 2.16.1 link	
Upgrade Meltano version	
TBA Data Platform	Current version
released > 1 year	

dbt packages inventory

package name			
Version in use DRI User			

----- ----- -----			
snowflakespend	1.1	N/A	-Data Engineers -Analytics Engineers
data-tests	N/A	N/A	-Data Engineers -Analytics Engineers
dbt-labs/audithelper	0.9.0	N/A	-Data Engineers -Analytics Engineers
dbt-labs/dbtutils	1.1.1	N/A	-Data Engineers -Analytics Engineers
dbt-labs/snowplow	0.15.1	N/A	-Data Engineers -Analytics Engineers
dbt-labs/dbtexternaltables	0.8.7	N/A	-Data Engineers -Analytics Engineers
brooklyn-data/dbtartifacts	2.8.0	N/A	-Data Engineers -Analytics Engineers

Approach to update tools version

Example for the end-of-life policy:

Regarding the upgrade policy, think we should at the minimum abide by the end of life policy. This is because anything becomes deprecated usually no longer receives any bug fixes, even critical security ones, which is a security risk to us.

- > 1. the airflow life cycle, end of life for 1.10.15 was June 17, 2021 so going forward, we should try to upgrade much earlier if following this end of life rule
- > 1. dbt latest releases
 - > 1. v1.2 and v1.3 are already at their end of life and will be completely deprecated end of 2023
 - > 1. dbt describes best practices for upgrading, one of them being to at least upgrade to new patch versions, which are "bugfix" or "security" releases

Libraries inventory list

The libraries inventory list is a bit automated process where we have a command line application that gathers all the information we need for upgrading libraries. Besides gathering all information about libraries, the application generates 2 reports that can be attractive for the upgrade process:

1. Get inventory list for each image - For utilities to check if the library we are

implementing is outdated and how far away from the latest version.

2. Check duplicated versions among images - To check if there is the version is in use in more than one image and the version is not in sync. If there is a version of libraries not in sync, possibly there is a specific reason why they are mismatched. Generally, it will be handy to keep the unique version across images (if possible and not a stopper).

To be able to run this program, checkout the /packageinventory repo and run the following code:

- Run program

```
bash
run file
python3 gitinventory.py [--dry-run] [--logging [print/logging]] [--reportfolder DEFINEFOLDER]
[--logfile DEFINELOGFILE] [--help]
```

For more details how the program is running, refer to the source code.

Note: Keep in mind that we are relying on PyPi and GitLab Data group as a primary source of the latest version of the library.

> Table 3: DRI for Python libraries

Tool name	DRI
Python libraries	@rbacovic

Approach to updating libraries

Proposal for creating a candidate list for upgrading criteria:

> Table 4: Example criteria for upgrading Python, tools or libraries version

Criteria	Risk for the implementation
Example (1.0 low, 5.0 high)	
----- -----	
----- -----	

The current version is outdated and not supported anymore (the end-of-life policy)	Python 2.7 4.0
The current version is vulnerable	
Article N/A	
Major version released	
Current version: 2.1.0	Latest version 3.0.0 3.0
Minor version released	
Current version: 2.1.0	Latest version 2.2.0 1.0

Patch version released	
Current version: 2.1.0 Latest version 2.1.8	1.0

As the library updating can be difficult to determine when and why to upgrade, a few considerations can assist us here. We should think about upgrade (in addition to the end-of-life criteria):

- Need a new feature for a particular version - in case there is a significant feature we need/want to use, should go for the upgrade
- Upgrade only if serious vulnerability in package - this is always a red flag and we should start upgrading immediately
- Upgrade if behind more than x major/minor versions - probably good reason to move on with the upgrade, with serious consideration of the impact. For instance, jumping to the newer major version, in some cases requires a new version of Python. If the risk is assessed properly, should move on

Dependency check up

The reason for upgrading can be dependent tool/package. For instance, if you plan to upgrade dbt and sometimes it requires upgrading · Python as well (for dbt-image image). That can be the reason to upgrade Python as it is required by other tools (dbt in this example).

Tips and tricks for upgrading

· Do`s:

- Do the extraordinary upgrade when there is a known issue that can impact our platform
- When upgrading the · Python package to a major version change, check compatibility with the Python version active in the image as there can be a collision with it
- Check if a · Python package you want to install/upgrade is safe and does not contain malicious code
- Always check backward compatibility with other packages (pipelines in the project will help you here during the image build).

· Dont`s:

- Do not upgrade to the pre-release version of the software, always use a stable release version
- Do not use non-trusted sources. We prefer PyPi or packages from GitLab Data group for the source of the installation

Upgrade logging

> Table 5: Log activities regarding the upgrade

Quarter (In which quarter we do upgrade planning)	Issue for check upgrades (Put the link to the issue you plan to use for the upgrade planning)	Upgrade execution (In which quarter we do upgrade execution)	Epic for planned upgrade (Put the link to the epic you plan to use for the upgrade execution)	Type of
--	--	---	--	---------

RStudio is an integrated development environment (IDE) for R that is available in both open source and commercial editions. RStudio is developed by Posit, a company that creates open source software for data science, scientific research, and technical communication. They are also responsible for many R resources and package development. Take a look at the resources page on the Posit website for more information, but below are just a couple of useful resources from RStudio.

- Books
- Tidyverse Blog
- Posit Blog
- RMarkdown Documents
- A ModernDive into R and the Tidyverse - This book is extremely helpful to beginners explaining the difference of R and RStudio and getting familiar with how to use RStudio

Before you download RStudio you must first download R. RStudio will not run if you have not downloaded R on your machine.

To download RStudio, go to the Posit website and navigate through Products > RStudio IDE > (click) Download RStudio > Download RStudio Desktop.

Or you can just go here and follow the steps on their website.

NOTE R and RStudio may be used interchangeably throughout this page.

Download and Configure Snowflake Driver (MacOS)

RStudio can connect to various database for production development of models or ad hoc analysis. If you want to connect to Snowflake below are some steps to accomplish this.

1. First you will need install unixODBC using homebrew. If you have not already installed homebrew on your machine, the website will give you the commands to do so. Once homebrew is installed, unixODBC can be installed using the command: `brew install unixodbc`
 - Alternatively, iODBC can be used, but this documentation uses unixODBC as the chosen driver manager.

1. This will create two configuration files, `odbcinst.ini` and `odbc.ini`.
 - `odbcinst.ini` holds the ODBC drivers information.
 - `odbc.ini` holds information required to connect to databases, such as host, username, etc.This is where you set up your DSN for your system.
 - to see the location of these configuration files, run the command `odbcinst -j`.

1. Download the latest driver for Snowflake here. You can then follow these instructions to complete the configuration of the driver on your machine.
 - As many parameters as desired can be entered in the configuration files, such as role, database, warehouse, username, etc. However, these can also be specified in RStudio. If you choose to set up the configuration files with these details, it may be necessary to set up a DSN for every database/schema used in Snowflake.
 - Below are examples of how to configure the `odbc.ini` and `odbcinst.ini` files in the user file location.

odbcinst.ini (location of file based on output from the odbcinst -j command above)

text

[Snowflake]

Driver = /opt/snowflake/snowflakeodbc/lib/universal/libSnowflake.dylib

odbc.ini (location of file based on output from the odbcinst -j command above)

text

[ODBC Data Sources]

SnowflakeDSII = Snowflake

[SnowflakeDSII]

Server = gitlab.snowflakecomputing.com

Port =

UID =

Schema =

Warehouse =

Driver = /opt/snowflake/snowflakeodbc/lib/universal/libSnowflake.dylib

Description = Snowflake DSII

Locale = en-US

Tracing = 0

Authenticator = gitlab.okta.com

This video shows the basic steps to connect a tool (it covers Excel) to Snowflake via ODBC.

Connecting to Snowflake in RStudio

The next step is to connect RStudio to Snowflake using the driver configurations you've just set up. This can be accomplished by using the DBI, tidyverse, and odbc packages in R. For a general overview on how to connect to databases in RStudio, please refer to this website for detailed information.

This is an example of the code that can be used to connect to Snowflake in R.

json

```
con <- DBI::dbConnect(odbc::odbc(),
  driver = "Snowflake",
  uid = rstudioapi::askForPassword("Database UserID"),
  role = [your user role],
  warehouse = [warehouse you wish to connect to],
  pwd = rstudioapi::askForPassword("Database password"),
  Authenticator = "externalbrowser",
  database = [database you wish to connect to],
  schema = [schema you wish to connect to],
  server = "gitlab.snowflakecomputing.com"
)
```

Some details regarding the above code:

1. DBI is a package that helps connect R to various databases. Above, we are using the `dbConnect()` function to pass our database parameters.
1. `odbc::odbc()` tells the function you are going to use an ODBC driver for this connection.
1. The `rstudioapi::askForPassword("")` function prompts the user to enter their UID and/or their PWD so it is not stored in their script.
1. `driver = "Snowflake"` is specific to the `odbcinst.ini` file set up above. This specifies which driver will be used to connect. (NOTE: if you are experiencing issues connecting, try changing the syntax to the actual path of the driver in R. Example: `driver = "/opt/snowflake/snowflakeodbc/lib/universal/libSnowflake.dylib"`).
1. `server = "gitlab.snowflakecomputing.com"` is specific to the snowflake instance being accessed.

OKTA Authenticator

Since OKTA or other authenticators are often used to connect to Snowflake, we have reference Authenticator several times in the directions above. The first is in the `odbc.ini` file, specifying the authenticator used here at GitLab (OKTA). It is then referenced in the parameters used with `DBI::dbConnect()` in the line `Authenticator = "externalbrowser"`.

The "externalbrowser" lets `dbConnect()` know it should reference the url specified in the configuration file to login to Snowflake. The password that is entered during the `rstudioapi::askForPassword()` prompt should be the users OKTA password.

Once you've completed the steps above and try running the code, you should be taken to a webpage to complete login. The console in R should display the following text before it takes you to the webpage.

Initiating login request with your identity provider. A browser window should have opened for you to complete the login. If you can't see it, check existing browser windows, or your OS settings. Press CTRL+C to abort and try again...

NOTE: You will have to stay on the webpage until it indicates your identity was confirmed and you were connected to Snowflake.

Managing R with .Rprofile

It is recommended to set up a `.Rprofile` file to customize the startup process for a given session in RStudio. It can also simplify sharing code with other users. Upon startup, R and RStudio will look for and run the `.Rprofile` file which can be used to control the behavior of your R session (e.g. setting options or environment variables).

`.Rprofile` files can be either at the user or project level. User-level `.Rprofile` files live in the base of the user's home directory, and project-level `.Rprofile` files live in the base of the project directory. R will source only one `.Rprofile` file. So if you have both a project-specific `.Rprofile` file and a user `.Rprofile` file that you want to use, you explicitly source the user-level `.Rprofile` at the top of your project-level `.Rprofile` with `source("~/Rprofile")`.

One easy way to edit your .Rprofile file is to use the `usethis::editrprofile()` function from within an R session. You can specify whether you want to edit the user or project level .Rprofile.

Follow the example below to set up a new .Rprofile file that automatically sets your username, role, and driver for snowflake. If other users follow the same template, they will not have to update this information when they access Snowflake (or any other database) tables using your code in R:

- Start by creating a blank .Rprofile document by installing packages and running the `editrprofile()` function from the `usethis` package

```
r
install.packages("usethis")
library(usethis)
usethis::editrprofile()
```

- In the .Rprofile file that opens in a separate tab enter in the necessary information:

```
r
.First <- function() cat("Welcome to R!")
.Last <- function() cat("Goodbye!")

uid = "CSMITH@GITLAB.COM"
role = "CSMITH"
driver = "/opt/snowflake/snowflakeodbc/lib/universal/libSnowflake.dylib"
styler::tidyversestyle()

message(" Successfully loaded .Rprofile ")
```

- Save the .Rprofile file. To test if it worked, at the top of the screen in R, navigate to Session >> Restart R

- Once R has restarted the message should show up in the console. In the example above this will be ` Successfully loaded .Rprofile`

Welcome to R!

- You will also see the variables `uid`, `role`, and `driver` in your environment. These variables are used for connecting to your database (Snowflake here at GitLab), or for any other variables you deem necessary.

dbplyr

The `dbplyr` package can be used to interact with databases using the tidyverse language. If you're familiar with tidyverse already, you may find this package especially helpful.

How to Use Git with RStudio

This documentation was creating using RStudio version 2022.07.1.

Objectives

1. Set up and install Git
2. Set up Git in RStudio
3. Clone an existing project from GitLab
4. Troubleshooting

Part 1: Installation and Setup

- Download and install R (if not already installed).
- Download and install RStudio Desktop (if not already installed).
- Install Homebrew (if not already installed).
- Install Git
 - Once Homebrew is installed, open your terminal (Command+Space Bar on Mac to open search bar, and search "Terminal")
 - Run the command `brew install git` in your terminal
 - Alternatively, Git can be downloaded [HERE](#). Make note of the path you install it to if you use this method.
- You will also need to have your GitLab account set up and access to the project you want to clone

Part 2: Setting Up Git in RStudio

- Open RStudio and go to Tools > Global Options > Git/SVN
- Check the box labeled Enable version control interface for RStudio project
- Set the path to the Git executable that you just installed.
 - If you don't know where Git is installed, access your Terminal and enter command `which git` and hit the return key
 - The path should be something similar to `/usr/bin/git`. (Note: if navigating through Finder, hidden files can be viewed by pressing Command + Shift + .)
- Create an SSH key by following the instructions [it/data-team/](#) the Generate an SSH Key Pair section.
 - ED25519 is recommended
 - Once complete, add the private key path to the SSH RSA Key field
 -
- Configure Git by setting your GitLab user name and GitLab email in RStudio
 - To open the Git prompt go to Tools > Shell and enter the following:
 - `git config --global user.name 'yourGitHubUsername'`
 - `git config --global user.email 'name@provider.com'`
- Restart RStudio

Part 3: Create an RStudio Project with Git

- To create a new project based on a remote Git repository:
 - Select File > New Project > Version Control
 -
 - Choose Git, then provide the repository URL:
 -
 - Access the GitLab project you want to clone
 - Select the Clone drop-down button at the top right

- Copy the URL for Clone with HTTPS
-
- Paste this link into the Repository URL section in RStudio
- Select Create New Project
- The GitLab Project should now be visible in R Studio
- Source for Walkthrough Instructions

Part 4: Troubleshooting

- Error:

```
console
Cloning into 'repo-name'
gitlab.com: Permission denied (publickey).
fatal: Could not read from remote repository.
```

Please make sure you have the correct access rights and repository exists.

- Solution: This is a known issue in certain versions of RStudio that is working to be resolved. Reinstalling an older version should resolve the issue (Source).

How to Update a GitLab Project with Updates from R Studio

- Before uploading changes made locally to a GitLab project ensure that you are working with the most current branch by selecting Pull with Rebase from the Git section in R (Ensure that you are rebasing from the main branch)
-
- Once changes are complete and ready to be uploaded select the new branch icon and enter a name for the branch (no spaces allowed). Select Create
-
- In the Review Changes window that opens in R ensure that changes on the left side of the screen are checked for Staged and that a commit message is entered on the right side of the screen.
- Select Commit
- In GitLab, navigate to the project you have made updates to. You should see a merge request that needs to be created and it will have the changes you made in R. Select the relevant reviewers and approvers to merge the changes.

How to Connect RStudio and Google Sheets

Google Sheets and R have the ability to interact via the googlesheets4 and googledrive packages in R.

1. Google App Authentication Setup
2. Package Installation
3. Reading Existing Google Sheets
4. Writing to Google Sheets

Part 1: Google App Authentication Setup

1. Follow the steps in the handbook to add yourself to the Google Cloud Project.
2. Submit and issue similar to this one to set up access for yourself specifically
3. Once the access is set up, you can use the below code to work through setting up and configuring access in RStudio.

```
r
library(google sheets4)
library(google drive)
google sheets4 Test
googleapp <- httr::oauthapp(
  "R",
  key = "[KEY].apps.googleusercontent.com",
  secret = "[SECRET]"
)

googlekey <- "[GOOGLEKEY]"

google sheets4::gs4authconfigure(app = googleapp,
  apikey = googlekey)

google sheets4::gs4authconfigure(client = gargle::gargleoauthclientfromjson("~/Google
Drive/Shared drives/People Analytics/Google API in R/google sheetsapis.json"),
  apikey = googlekey)

google sheets4::gs4auth()

Test Read
google sheets4::readsheet(ss = "https://docs.google.com/spreadsheets/d/1Oe7AduRIKO7Zqh60v51Zn-
WnTJYpcMDho394urBmpA",
  sheet = "stopwords") |>
  View()

googledrive Test

googledrive::driveauth()

googledrive::drivemv(file = "[SHEETNAME]",
  path = asid("[PATH]"),
  overwrite = TRUE)
```

Part 2: Installation

- Run the following code in R to install the necessary packages in RStudio

```
r
pkg <- c("google sheets4", "googledrive")
invisible(lapply(pkg, function(x) if (x %in% rownames(installed.packages()))==F)
```

```
install.packages(x)))  
invisible(lapply(pkg, library, character.only = TRUE))  
rm(pkg)
```

Part 3: Reading Existing Google Sheets

- The `readsheet()` function will allow you to read an existing spreadsheet
 - Run the `readsheet()` command in R pointing to the Spreadsheet URL you want to view
 - URL example: `googlesheets4::readsheet("https://docs.google.com/spreadsheets/...")`

- When you first try to access a spreadsheet you will be prompted to enter your account information
 - Enter Yes in RStudio when asked "Is it Ok to cache OAuth access credentials in the folder between R Sessions"
 - You will then be prompted to log into your Google Account in the browser
 - Check the box to allow the Tidyverse API Packages to access Google Sheets spreadsheets
 - A new window will open saying authentication is complete. Close the browser window.
 - rerun the `readsheet()` command again to confirm you can see output in R

Part 4: Writing to Google Sheets

Below are a list of functions that can be used to write data into a Google Sheet with examples.

- `gs4create()` can create a new spreadsheet and optionally populate initial data
 - example:

```
r  
(ss <- gs4create("fluffy-bunny", sheets = list(flowers = head(iris))))
```

- `sheetwrite()` (over)writes a whole data frame into a tab within a Google Sheet.
 - example:

```
r  
head(mtcars) %>%  
sheetwrite(ss, sheet = "autos")
```

- `rangewrite()` writes/overwrites a data frame into the same range of cells in a Google Sheet. Target sheet must already exist.
 - example:

```
r  
df <- dataframe  
ss <- "https://docs.google.com/spreadsheets/..."  
googlesheets4::rangewrite(ss = ss,  
data = df,  
sheet = "tabname")
```

- rangeclear() can be used to clear data from an existing spreadsheet tab
- example:

```
r
df <- dataframe
ss <- "https://docs.google.com/spreadsheets/..."
googlesheets4::rangeclear(ss = ss,
  sheet = "tabname"
  range = "tabname!A2:ZZ1000000")
```

- sheetappend() can be used to add rows to an existing tab. NOTE: this function will exclude column headers as a row in the target sheet.

- example:

```
r
df <- dataframe
ss <- "https://docs.google.com/spreadsheets/..."
googlesheets4::sheetappend(
  ss = ss,
  data = df,
  sheet = "tabname")
```

- Source for more information on this topic.

SECTION: enterprise-data/platform/safe-data/index.md

What is SAFE Data?

The Data Team follows the GitLab SAFE Framework when it comes to SAFE Data. Sometimes also MNPI and RESTRICTEDSAFE is used in relation to SAFE Data and the SAFE Framework.

Access to SAFE Data

Tableau

Access to Tableau dashboards is based on license type (see Tableau Licenses in our Tableau handbook page), which can be prioritized by job roles, and governed by the SAFE Data Access Framework. In Tableau, we apply the SAFE security framework using User Groups. There are four primary projects/folders where dashboards can exist - Development, Production, Ad-hoc, and Resources. The Development, Production, and Ad-hoc folders each have a sub-folder for various teams at GitLab, such as Finance and Sales. Within each team's folder there are generally two main folders to which content can be published: General, and SAFE.

- The General folders contain data that can be viewed by anyone at GitLab.

- The SAFE folders contain any dashboards with material non-public information (MNPI) and are only accessible by users belonging to the General SAFE Access user group.

The development workflow for Tableau Creators that work with data meeting GitLab's SAFE criteria is to publish into the SAFE folder in their department's Development project. Should the Creator want to publish the workbook outside of the SAFE folder within the development project, they should open an issue using the All Requests template in the Tableau project and request a review of the content by the BI Platform team. The BI Platform team will provide a yes or no decision on whether it is SAFE to publish the content outside of the SAFE folder in the development project.

Accessing a GitLab General Access Dashboard

General Access Dashboards are available to all GitLab Team members. An Access Request to gain access to Tableau (if a team member does not already have access) will be required, but no further Access Requests are required to view these general access dashboards. You can read more about permissions and access in the Tableau Section of the handbook.

Accessing a SAFE Dashboard

All SAFE dashboards are stored within various team's SAFE folders and permission is managed by the user's access group membership. Access to a single SAFE dashboard provides access to all SAFE Dashboards. Having access to the SAFE project/folder will result in making the team member a Designated Insider subject to our Insider Trading Policy. Gaining access to a SAFE dashboard (and the space) via membership in the General SAFE Access user group requires:

1. Your immediate manager's approval

To gain access to SAFE dashboards:

1. Create an Access Request for Tableau - SAFE Access - [Requestor Name].
1. Request approval from your immediate manager. Approval is needed if it concerns a new Access Request(issue).
1. After processing is complete you will be able to log in to Tableau and access your requested SAFE dashboard and all other dashboards which require SAFE access.

Please see the Accessing SAFE Data in Snowflake for instructions on to access the SAFE data in Snowflake.

SAFE Data in Google Sheets files can be accessed using the instructions for Accessing a SAFE Dashboard. Please follow those instructions to gain access to SAFE Data in Google Sheets.

Deprovisioning SAFE Dashboard access

Every 90 days the Data Team runs a Data Health and Security Audit. In this audit the Data Team also checks for inactive usage. In case a GitLab Team Member hasn't used the SAFE Dashboard space for more than 90 days, access will be deprovisioned. To get access again to the SAFE Dashboard space, a new AR needs to be created and all approvals need to be obtained again.

Accessing Various Limited Access User Groups

There are several variations of limited access user groups which you may want to access, such as internal audit-related data, or sales development SAFE data. You can read about the descriptions of these groups here, as well as how to request access.

Snowflake

All SAFE Data are stored in tables and views within separate database schemas in Snowflake. Access to 1 table provides access to all SAFE tables. Access to SAFE data requires:

1. Your immediate manager's approval.

To gain access to SAFE Data:

1. Create an Access Request and provide your needs and intent.
2. Request approval from your immediate manager. Approval is needed if it concerns a new Access Request(issue).
3. Once the request is approved, tag the Snowflake provisioners and they will process the request.
4. After processing is complete you will be able to access SAFE Data (schemas) in Snowflake.

Data Development

SAFE Data in Snowflake is stored in separate schemas. Schemas with SAFE Data are prefixed with: RESTRICTEDSAFE. All models that contain MNPI data must be stored in these schemas. There is a CI-Pipeline (safemodelscript) that secures this process. Every model that is created or changed, will be checked by the CI-Pipeline. As a developer, the following governance needs to be taken into account:

- mnpi tag to be applied when reading out of a table/model that holds MNPI data and MNPI is used in the model. Subsequently the data must be stored in a RESTRICTEDSAFE schema.
- mnpiexception tag to be applied when reading out of a table/model that holds MNPI data and MNPI is not used in the model. I.e. only a NON-MNPI field is used from a MNPI model.
- It's up to the engineer to determine if and which tag needs to be applied. Check the data that is exposed in your model against the GitLab SAFE Framework. If there is doubt, reach out in the. SAFE channel in Slack.
- The CI pipeline is there to help and check fact based (upstream model tags versus current model tags).

This video provides an overview of the SAFE Data Program implementation on Snowflake.

The following diagram describes the process in detail when handling or referencing SAFE Data.

SECTION: enterprise-data/platform/snowflake/_index.md

What and why

Snowflake is our Enterprise Data Warehouse (EDW) and is the core technology in our Enterprise Data Platform.

What does Snowflake Contain?

Snowflake contains all of our analytical data and Data Sources defines the set of original/raw data available.

Related Content

- Access
- Support Portal Access
- Compute Resources
- Data Masking
- Backups

Logging In

Login to Snowflake from Okta.

Navigating the UI

The Snowflake Quick Tour of the Web Interface provides comprehensive documentation for the UI.

Snowflake account configuration

ABORTDETACHEDQUERY

ABORTDETACHEDQUERY parameter is set on account level to True.

We often have cases where the connectivity was lost and the query keeps trying to run and still does not complete. It is meaningless for the query to keep running in these cases and adds no value. There is a grace period of 5 minutes. If the connectivity isn't fixed in 5 minutes, it stops the execution so the warehouses won't be running unnecessarily.

Data Freshness

We leverage automatic monitors in MonteCarlo for monitoring the freshness of the data.

SECTION: enterprise-data/platform/snowflake/clustering/index.md

Snowflake supports clustering, which is a powerful feature that allows users to optimize query performance by organizing data within micro-partitions. Clustering can significantly improve the efficiency of data retrieval, especially for large tables with frequently accessed columns.

How Clustering Works

Clustering in Snowflake works by:

1. Defining one or more columns as clustering keys
2. Reorganizing data within micro-partitions based on these keys
3. Maintaining the clustering order as new data is added or modified

This process helps Snowflake's query optimizer to quickly locate and retrieve relevant data, reducing the amount of data that needs to be scanned.

Benefits of Clustering

- Improved query performance
- Reduced compute costs
- Better pruning of irrelevant micro-partitions
- Enhanced data organization

Costs of Clustering

- Substantial upfront cost whenever the table is clustered (usually, this happens on full-refresh)
- Marginal maintenance costs for ongoing updates to maintain the clustering order

When to Use Clustering

If a model includes the `incrementalbackfilldate` variable then it must also be configured to never fully refresh. These models can incur large re-clustering charges since the backfill DAG reprocesses months in random order which leads to an unsorted table. An example of this is `martbehaviorstructuredevent`.

Models which can be fully refreshed without additional configuration can be clustered in the normal way as dbt will produce a clustered table.

Consider using clustering when:

- Your table is large (typically > 1 TB)
- You frequently query on specific columns
- Your queries are experiencing performance issues in dbt, Snowsight, or Tableau; clustering the table that your query selects from can improve query performance.
- You have high cardinality columns that are often used in WHERE clauses or JOINS

How to Implement Clustering

dbt syntax

```
sql
clusterby=['column1','column2'],
automaticclustering='true'
```

The cluster key and `automaticclustering` configuration setting must be added to the model so that automatic clustering is enabled by default should the model be rebuilt. A full-refresh of an incremental model will be required before the automatic clustering is enabled.

Manual applying via SQL

Manual clustering of a table can be achieved by executing the following SQL command. However, it's important to note that this method is not recommended for dbt models. The reason is that if the table is dropped and recreated through dbt, the clustering will not persist. For dbt models, it's advisable to use the dbt code mentioned earlier to ensure proper and persistent clustering:

```
sql
ALTER TABLE yourtable CLUSTER BY (column1, column2, ...);
```

Automatic-clustering needs to be enabled using the RESUME RECLUSTER command. For example:

```
sql
ALTER TABLE sales CLUSTER BY (date, region) RESUME RECLUSTER;
```

Monitoring Clustering

Snowflake provides system functions to monitor clustering:

```
sql
SELECT SYSTEM$CLUSTERINGINFORMATION('yourtable');
```

This function returns valuable information about the clustering state of your table. Some examples of how to interpret the information can be found in the Snowflake docs.

Ideally, `averageoverlaps` would be below 1 and `averagedepth` would be ~ 1. A high number indicates the table is not well clustered.

Best Practices

1. Choose clustering keys wisely based on your query patterns
2. Monitor clustering regularly
3. Reconsider clustering if your table is small or experiences frequent updates
4. Be aware of the additional storage and maintenance costs associated with clustering
5. When a table with auto-clustering enabled is cloned, the cloned version will have auto-clustering turned off

By understanding and properly implementing clustering, you can significantly enhance the performance of your Snowflake queries and optimize your data warehouse operations.

SECTION: enterprise-data/platform/snowflake/snowpipe/index.md

Overview

Snowpipe can push error notifications to a cloud messaging service when it encounters errors while loading data i.e. through snowpipe or on failure of snowflake task. The notifications describe the errors encountered in each file, enabling further analysis of the data in the files for snowpipe. Task error notifications trigger a notification describing the errors encountered when a task executes SQL code. The notifications describe any errors encountered during task execution.

Currently, cross-cloud support is not available for push notifications. Configure error notification support for the messaging service provided by the cloud platform where your Snowflake account is hosted.

Since our Snowflake is hosted in AWS the setup for push notification has been done in AWS.

For both snowpipe and snowflake task the SNS integration is same.
Over here enabling-error-notifications Snowflake has defined steps for the same.

Snowflake SnowPipe and Tasks Triage

Notified a repeatable situation occurred:

- Occasionally we see a drop in data and notify it after some time - we implemented mechanism to have a push warning about missing or incomplete data. Usually, we use a reactive approach after we are informed from the business side, which is not the preferred way. Now, we are more proactive and alerted immediately when error happened.
- The pipelines we monitor (versiondb, Snowplow, PTO) are running either via Snowflake tasks or Snowpipe mechanism.
- Used a comprehensive approach to focus and reveal shape and status of pipelines inside Snowflake

Workflow

```
mermaid
graph LR
    PM[Pipelines monitoring]
    subgraph DE
        TM[Technical monitoring]
        PH[Pipeline health]
        DL[Data freshness check]
    end
    subgraph DEAE[DE+AE]
        OD[Observing data]
        MT[Metrics throughput check]
    end
    PM-->TM
    TM-->PH
    TM-->DL
    PM-->OD
    OD-->MT
```

Technical monitoring

Have to make sure that we get notified via Slack if there are errors on SnowPipe and/or Snowflake tasks. In case of an error and if it turns out that the root cause is upstream we have to solve in collaboration with source owners. The Data Platform is responsible for health of pipelines.

```
mermaid
graph LR
    SNSSETUP
    Lambda-->SLACK
    SNSSETUP-->SNSTOPIC
    subgraph Snowflake
        TASK(Tasks)
        SNOWPIPE(Snowpipes)
        SNSSETUP[SNS setup for Taks and Snowpipes]
        SNOWPIPE-->SNSSETUP
        TASK-->SNSSETUP
    end
    end

    subgraph AWS [AWS]
        SNSTOPIC[SNS Topic] --> Lambda[Lambda function Python]
    end
    end

    subgraph Slack
        SLACK[data-pipelines channel alert]
    end
    end
```

Below covered details of the name and commands used for setup.

Step 1: Created SNS topic in AWS named gitlab-snowflake-notification following the instruction.

For this full access on SNS topic should be provided to the user
Same SNS topic can be used for all snowflake task and snowpipe push notification.
Capture the ARN value for the topic as it will be required in step 4.

Step 2: Requested IT help for Creating the IAM Policy in AWS

It Create an AWS Identity and Access Management (IAM) policy that grants permissions to publish to the SNS topic.

Step 3: Requested IT help for Creating the AWS IAM Role in AWS.

Create an AWS IAM role on which to assign privileges on the SNS topic
Record the Role ARN value located on the role summary page.

Step 4: Creating the Notification Integration in snowflake

To create Notification Integration ACCOUNTADMIN privileges are required.

```
sql
CREATE OR REPLACE NOTIFICATION INTEGRATION gitlabdatanotificationint
  ENABLED = true
  TYPE = QUEUE
  NOTIFICATIONPROVIDER = AWSSNS
  DIRECTION = OUTBOUND
  AWSSNSTOPICARN = '<topicarn>'
  AWSSNSROLEARN = '<iamrolearn>';
```

Step 5: Granting Snowflake Access to the SNS Topic

Run below query in Snowflake:

```
sql
DESC NOTIFICATION INTEGRATION gitlabdatanotificationint;
```

And capture SFAWSIAMUSERARN and SFAWSEXTERNALID post this modify the Trust Relationship in the IAM Role by setting

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "",
      "Effect": "Allow",
      "Principal": {
        "AWS": "<sfawsiamuserarn>"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "<sfawsexternalid>"
        }
      }
    }
  ]
}
```

This will require the IT help to update the policy.

Step 6: Grant usage permission on integration to loader role

With ACCOUNTADMIN role execute below in Snowflake.

sql

GRANT USAGE on INTEGRATION gitlabdatanotificationint to role loader;

Note: Above setup should not be modified because modifying any step will require all the step from 1 to 6 redo as the integration will break.

Step 7: Create Lambda function to send slack notification

For this the user should have Create lambda function and ListRoles permission in AWS. If any of those 2 privileges are missing, create an AR issue.

For Creating lambda function select the following :

1. Author from scratch
1. Give unique name to lambda function
1. Select any of the Python version (everything ≥ 3.10 is fine)
1. Architecture Default
1. it/data-team/ Change default execution role select Use an existing role and it/data-team/ this select Gitlab-lambda-snowflake
1. Click Create function button

More details are shown in the image below.

Once the function is created it/data-tea